

Alberi Binari

Alberi Binari e Alberi di Ricerca Binari, Operazioni di Visita

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio



Alberi Binari

Albero Binario



- **Albero Binario**
 - È una struttura di dati gerarchica in cui ogni nodo ha fino a due nodi figli
 - Struttura dati autoreferenziale (ricorsiva)
- Definizione astratta:
 - Un albero binario **bt** (di elementi di tipo **T**) è:
 - Q l'albero vuoto, indicato con **/**
 - Q una tripla **⟨left, e, right⟩** dove:
 - **e** è un valore (di tipo **T**)
 - Detto **radice** (**root**) di **bt**
 - **left, right** sono alberi binari (di elementi di tipo **T**)
 - **left** è detto **sottoalbero sinistro** (**left subtree**) di **bt**
 - **right** è detto **sottoalbero destro** (**right subtree**) di **bt**

Albero Binario: Definizioni

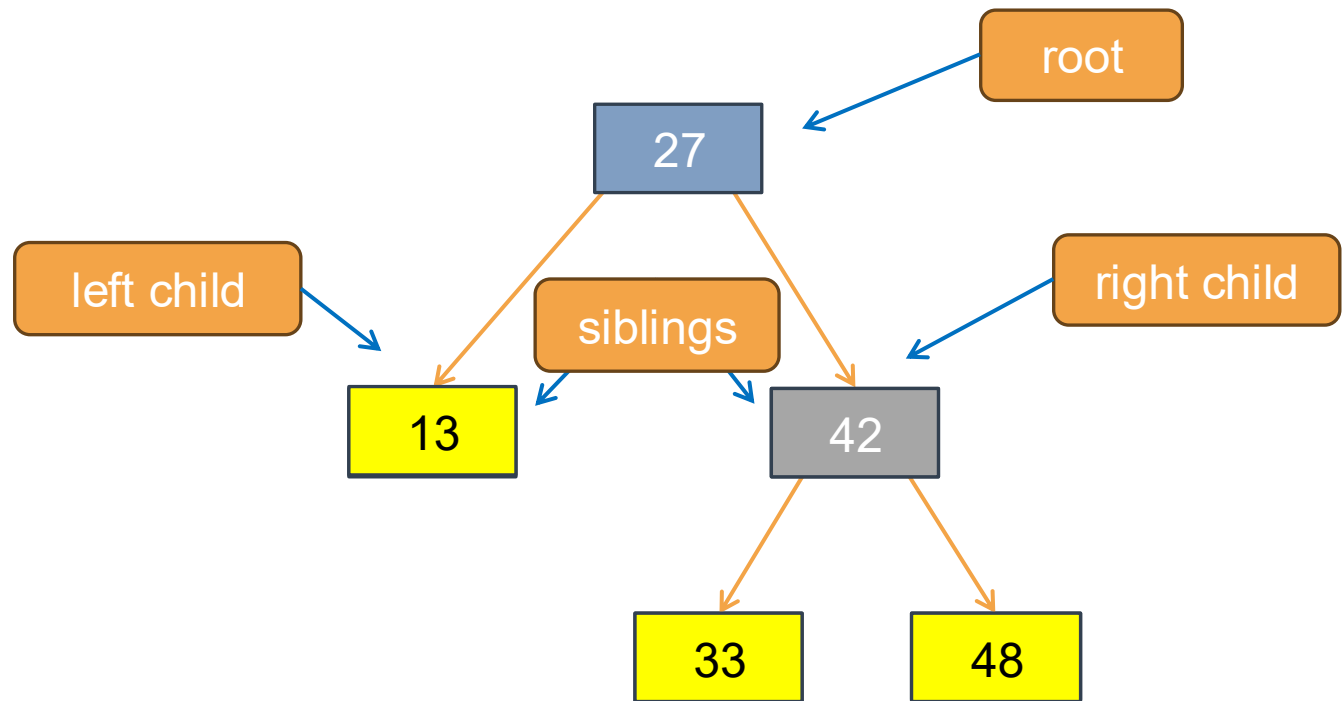


- **bt** dato da $\langle \text{left}, e, \text{right} \rangle$:

- L'elemento **e** è detto:
 - **radice** (**root**) di **bt**
- La radice di **left** è detta
 - **figlio sinistro** (**left child**) di **e**
- La radice di **right** è detta:
 - **figlio destro** (**right child**) di **e**
- I figli di **e** sono detti
 - **fratelli** (**siblings**)
- Un elemento senza figli è detto:
 - **foglia** (**leaf**)
- Un elemento è **interno** (**inner**):
 - Se non è né la radice né una foglia

- Esempio di albero binario **bt**:

- $\langle \langle /, 13, / \rangle, 27, \langle \langle /, 33, / \rangle, 42, \langle /, 48, / \rangle \rangle$



Albero Binario: In C



Da definizione astratta:

- Un albero binario **bt** (di elementi di tipo **E**) è:
 - **Q** l'albero vuoto, indicato con **/**
 - **Q** una tripla **<left, e, right>**:
 - **e** è un valore (di tipo **T**)
 - **left, right** sono alberi binari (di elementi di tipo **T**)

... a implementazione concreta in C:

- Una variabile **bt** (di tipo **Tree**, id est **struct treeNode***) rappresenta un albero binario (di elementi di tipo **E**) se contiene:
 - **Q** l'albero vuoto, indicato con il valore **NULL**
 - **Q** un puntatore ad una **struct treeNode** dove:
 - **data** contiene un valore (di tipo **T**)
 - **left, right** rappresentano rispettivamente alberi binari (di elementi di tipo **T**)

```
// Tipo per liste di (elementi di tipo) T
typedef ... T; // Il tipo degli elementi

typedef struct treeNode *Tree; // Il tipo degli alberi binari

struct treeNode {
    Tree left;
    T data;
    Tree right;
};
```

Usando typedef

```
// Tipo per gli alberi binari (senza typedef)

struct treeNode {
    struct treeNode *left;
    ... data;
    struct treeNode *right;
};
```

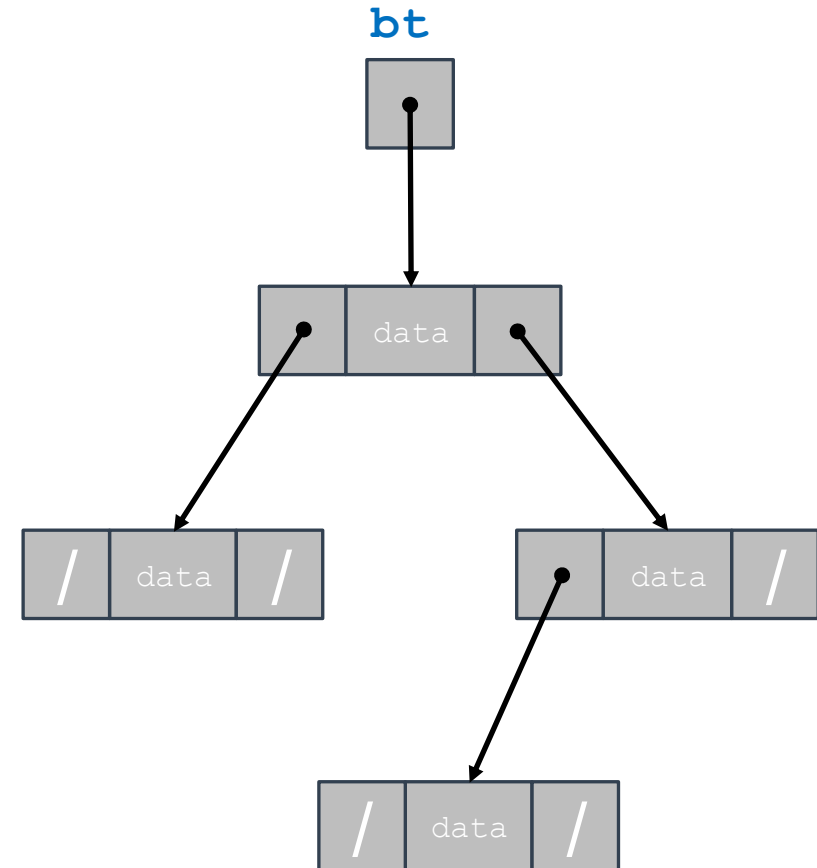
Senza typedef

Albero Binario In C: Esempio

```
// Tipo per liste di (elementi di tipo) T
typedef ... T; // Il tipo degli elementi
typedef struct treeNode *Tree;

struct treeNode {
    Tree left;
    T data;
    Tree right;
};
```

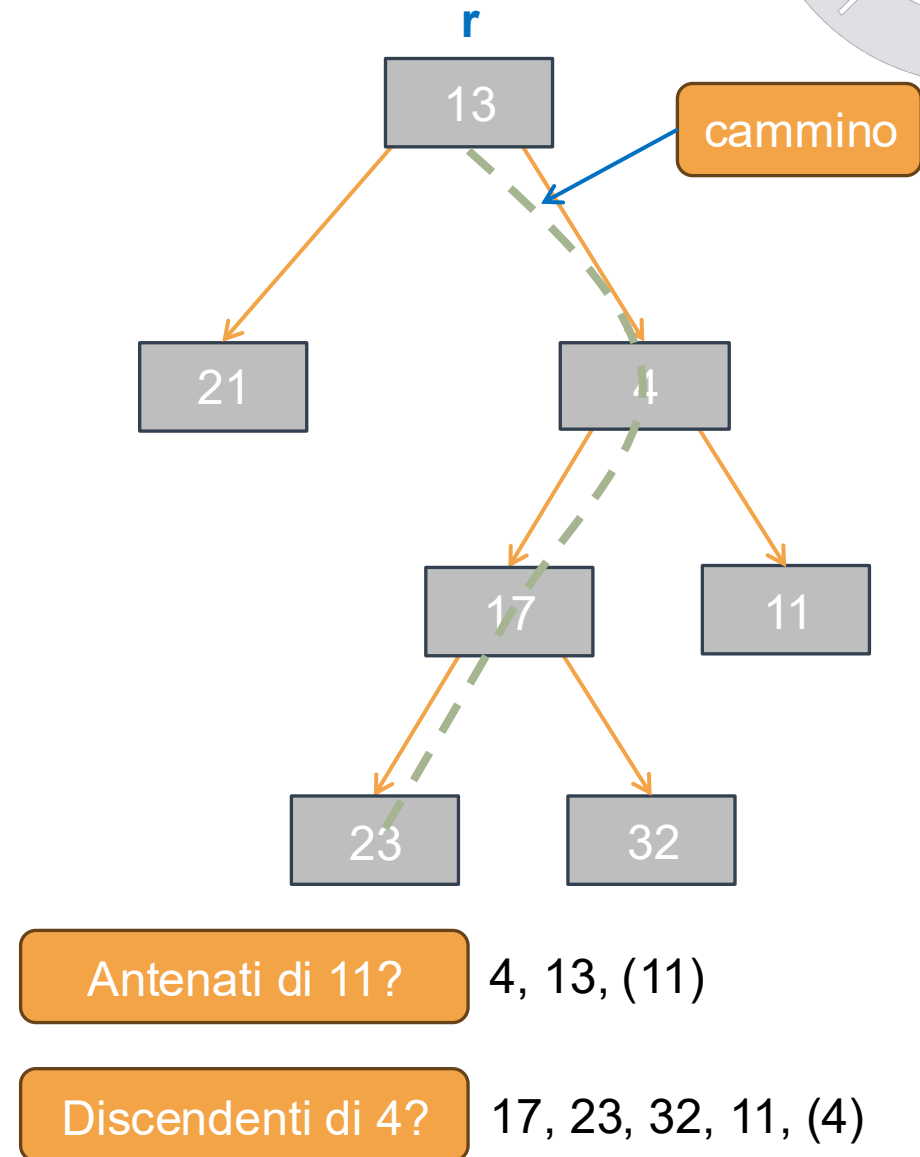
Usando typedef



Alcune Definizioni sugli Alberi Binari

Cammini in un Albero

- Un **cammino** da un nodo **n** ad un altro nodo **m**:
 - È una sequenza di nodi connessi da archi che portano da **n** ad **m**
- Dato un nodo **n** di un albero binario **T** con radice **r**
 - Un qualsiasi nodo **m** lungo un cammino dalla radice **r** ad **n** è detto **antenato** di **n**
 - Se **m** è antenato di **n**, allora **n** è **discendente** di **m**
- Ogni nodo è antenato e discendente di sé stesso
 - Se **m** è diverso da **n**, allora **m** è antenato proprio di **n**, e **n** è discendente proprio di **m**

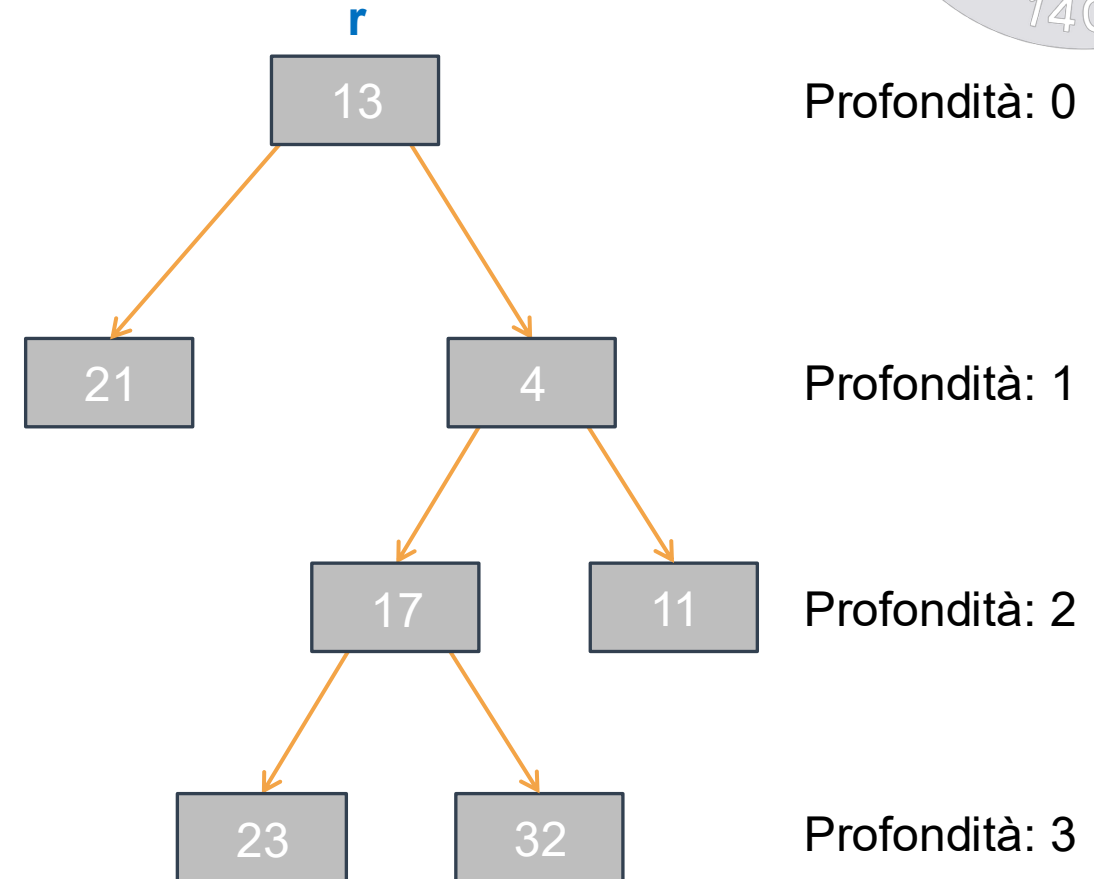


Profondità di un Albero

- In un albero la **profondità** di un nodo è la lunghezza del cammino che porta dalla radice a quel nodo
 - Cioè il numero di archi tra la radice ed il nodo
 - La radice ha quindi profondità 0
- L'**altezza** dell'albero è la profondità del nodo di profondità massima
 - L'albero vuoto ha altezza -1

Profondità di 11? 2

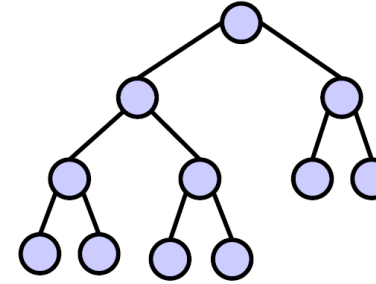
Altezza dell'albero? 3



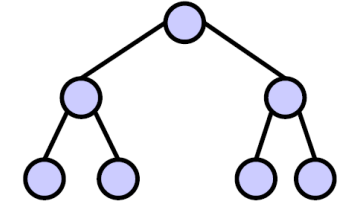
Grado di un Nodo e Completezza Albero



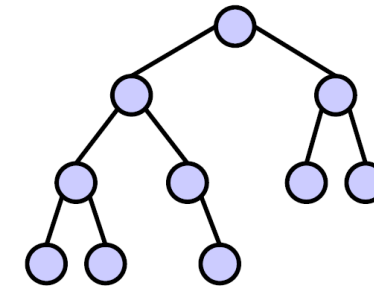
- Il **grado** di un nodo è il suo numero di figli
- Un albero binario si dice **completo** se:
 - Tutte le foglie hanno la stessa profondità
 - E tutti i nodi interni hanno grado 2
 - Cioè, hanno esattamente due figli
- Un albero binario è **pseudo quasi completo** se
 - Tutti i livelli, tranne al più l'ultimo, sono completi
- Un albero binario è **quasi completo** se
 - Le foglie mancanti nell'ultimo livello sono consecutive a partire dall'ultima a destra



(a)



(b)



(c)

Albero (a)?

Quasi completo

Albero (b)?

Completo

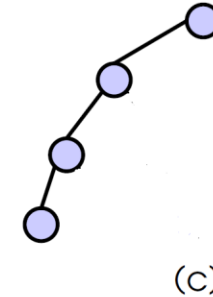
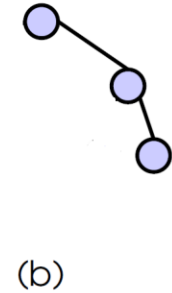
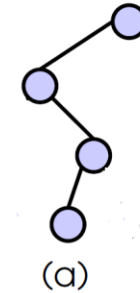
Albero (c)?

Pseudo quasi completo

Albero Degeneri e Completezza Sbilanciata



- Un albero binario con n nodi e altezza h si dice
 - Degenerare se ogni nodo ha al più un figlio
 - Da cui segue $h == n-1$
 - Completamente sbilanciato
 - a sinistra
 - se ogni nodo ha al più il figlio sinistro
 - a destra
 - se ogni nodo ha al più il figlio destro



Albero (a)?

Degenerare

Albero (b)?

(Degenerare e) completamente sbilanciato a destra

Albero (c)?

(Degenerare e) completamente sbilanciato a sinistra

Alcune Proprietà degli Alberi Binari

Numero di Nodi di Albero Binario Completo

Proposizione 1 (numero di nodi di un albero binario completo):

- Un albero binario completo di altezza h ha $2^{h+1}-1$ nodi

Dimostrazione. La prova è per induzione su h

- Caso base ($h=-1$): un albero binario completo di altezza $h=-1$ ha zero nodi (è vuoto). Siccome $2^{-1+1}-1 = 0$, il caso è verificato.
- Caso base ($h = 0$): Un albero binario completo di altezza $h = 0$ ha un solo nodo (la radice). Siccome $2^{0+1}-1 = 1$, il caso è verificato.
- Passo induttivo ($h > 1$): Assumiamo (per ipotesi induttiva) che un albero binario completo di altezza h abbia $2^{h+1}-1$ nodi, e dimostriamo che un albero binario T completo e di altezza $h+1$ ha $2^{(h+1)+1}-1 = 2^{h+2}-1$ nodi.
 - L'albero T ha (per definizione) la radice, più un sottoalbero sinistro T_s e un sottoalbero destro T_d entrambi completi e di altezza h . Per ipotesi induttiva $\#nodi(T_d) = \#nodi(T_s) = 2^{h+1}-1$.
 - Quindi:
 - $\#nodi(T) = \#nodi(radice) + \#nodi(T_s) + \#nodi(T_d) = 1 + (2^{h+1}-1) + (2^{h+1}-1) = 1 + 2 \cdot 2^{h+1} - 1 - 1 = 2^{h+2} - 1 = 2^{(h+1)+1} - 1$

Numero di Foglie di Albero Binario Completo



Proposizione 2 (numero di foglie di un albero binario completo):

- Un albero binario completo di altezza h ha 2^h foglie

Dimostrazione. Lasciata come esercizio.

Suggerimento: la dimostrazione è di nuovo per induzione su h

Numero di Nodi di un Albero Binario Quasi Completo



Proposizione 3 (numero di nodi di un albero binario quasi completo):

- Sia T un albero binario quasi completo di altezza h . Allora: $2^h \leq \#nodi(T) < 2^{h+1}-1$

Dimostrazione. Per dimostrare questo risultato abbiamo bisogno di determinare il numero massimo e minimo di nodi di un albero binario quasi completo di altezza h .

- Il numero massimo di nodi che un albero binario quasi completo di altezza h può avere è pari al numero di nodi dell'albero binario completo di altezza h , ossia
 - $\max = 2^{h+1}-1$ (vedi Proposizione 1)
- L'albero binario quasi completo di altezza h con numero minimo di nodi ha la forma come in figura:
 - Dove T_s e T_d sono degli alberi binari completi di altezza $h-2$
 - Allora: $\#nodi(T) = 1 + 1 + \#nodi(T_s) + \#nodi(T_d)$
 - Cioè: $\#nodi(T) = 2 + (2^{h-1}-1) + (2^{h-1}-1) = 2 \cdot 2^{h-1} = 2^h$

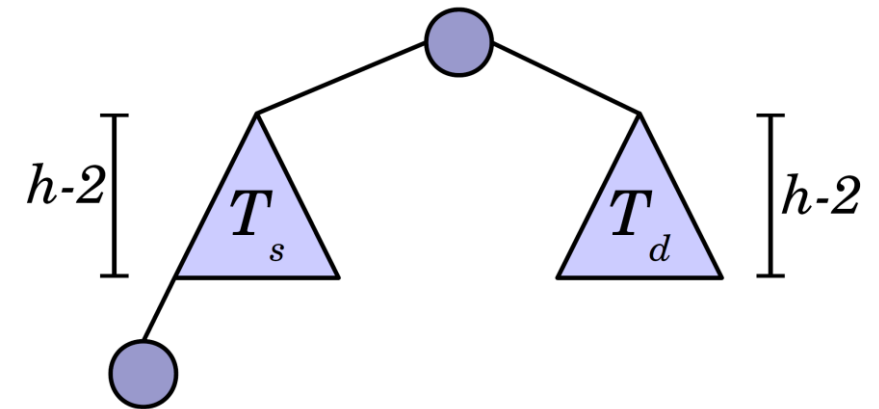


Figura: Albero binario quasi completo di altezza h con numero minimo di nodi

Numero di Nodi di un Albero Binario Quasi Completo



Proposizione 4 (altezza di un albero binario quasi completo):

- L'altezza di un albero binario quasi completo T con n nodi è $h = \lfloor \log n \rfloor$

Dimostrazione:

- Per la Proposizione 3 abbiamo che
 - $2^h \leq n \leq 2^{h+1}-1 < 2^{h+1}$
- quindi
 - $h \leq \log_2 n < h+1$, cioè $h \leq \lfloor \log_2 n \rfloor < h+1$
- da cui segue
 - $h = \lfloor \log_2 n \rfloor$

Principio di Induzione Strutturale sugli Alberi Binari

Principio di Induzione Strutturale sugli Alberi Binari



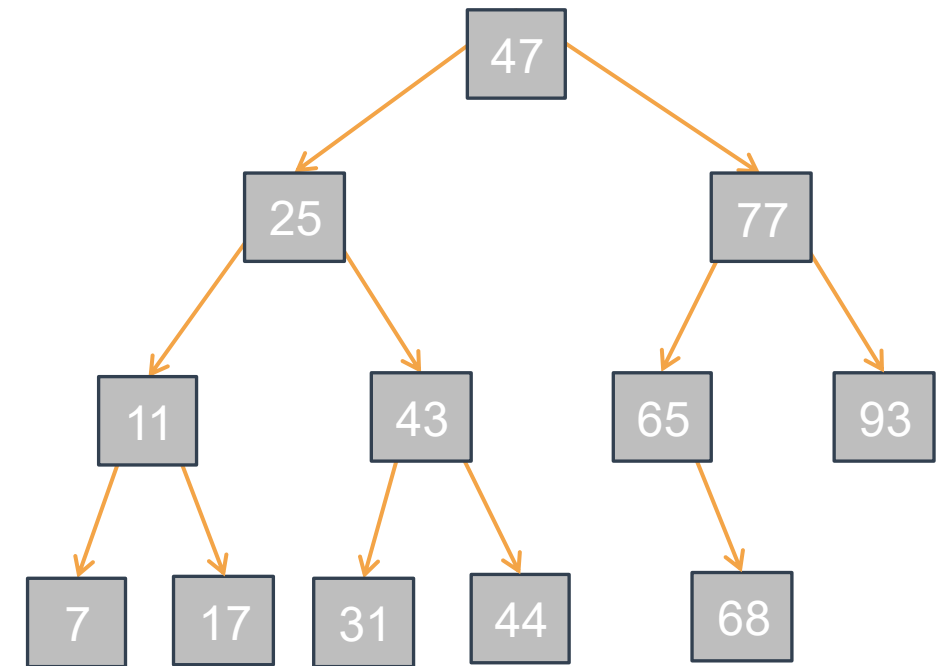
- Se, data una proprietà **P(bt)** sugli alberi binari,
 - Riusciamo a dimostrare che tale proprietà vale per l'abero vuoto **/** (**caso base**).
 - Assumendo che essa valga per gli alberi **left** e **right**, e, dato un qualunque elemento **e**, riusciamo a dimostrare che essa vale anche per l'albero binario **⟨left, e, right⟩** (**caso induttivo**).
 - Allora, abbiamo dimostrato che **P()** vale per qualunque albero binario.
-
- Il **principio di induzione su strutture dati ricorsive** (di cui Principio di Induzione Strutturale sugli Alberi Binari è un caso particolare) è una generalizzazione del principio di Induzione sui Numeri Naturali
 - Ricopre, nello sviluppo di algoritmi ricorsivi che operano su alberi binari, un ruolo analogo a quello ricoperto dalla **metodologia di sviluppo basata sull'invariante di ciclo** nello sviluppo di algoritmi iterativi.

Alberi di Ricerca Binari (ARB) Binary Search Trees (BSTs)

Alberi di Ricerca Binario (ARB)



- Un albero di ricerca binario è un albero binario **bt** tale che:
 - Gli elementi appartengono ad un insieme totalmente ordinato rispetto ad una relazione $\leq$ (minore o uguale)
 - E per ogni elemento **k** di **bt**:
 - Tutti gli elementi del sottoalbero sinistro di **k** sono $\leq k$
 - E tutti gli elementi del sottoalbero destro di **k** sono $\geq k$
- Se tutti gli elementi di **bt** sono distinti:
 - Si possono sostituire $\leq$ e $\geq$ con $<$ e $>$
 - (strettamente minore e strettamente maggiore)



Esempio Albero di Ricerca Binario di Numeri Naturali

Visita degli Elementi di un Albero Binario

Preliminari: Tipi di Alberi Binari Usati

- Definiamo tre tipi di alberi:
 - per `int`, `char` e generici

```
// Tipo per alberi di interi
typedef struct intTreeNode
    IntTreeNode, *intTree;

struct intTreeNode {
    intTree left;
    int data;
    intTree right;
};
```

Per `int`

```
// Tipo per alberi di char
typedef struct charTreeNode
    CharTreeNode, *charTree;

struct charTreeNode {
    charTree left;
    char data;
    charTree right;
};
```

Per `char`

```
// Tipo per alberi generici T
typedef struct treeNode
    TreeNode, *Tree;

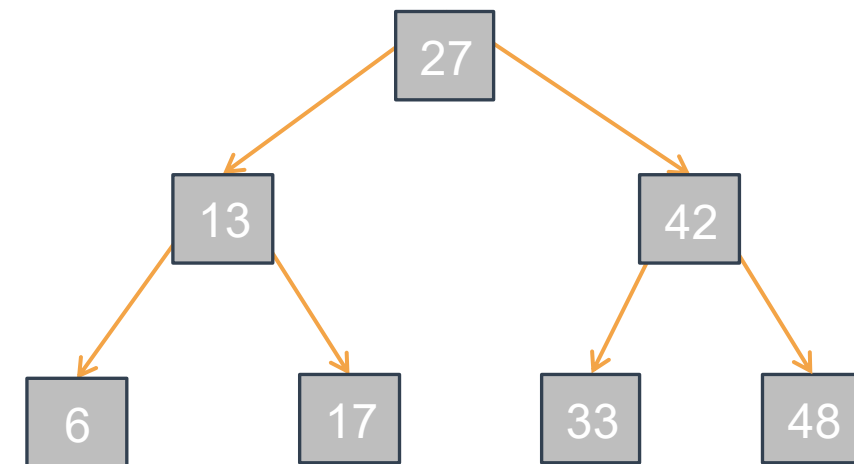
struct treeNode {
    Tree left;
    T data;
    Tree right;
};
```

Per tipo `T`

Visita: Algoritmi Ricorsivi

Tre algoritmi di visita ricorsivi che differiscono nell'ordine di visita radice e sottoalberi:

- Visita **in-order** – `void inOrder(Tree tree);`
 - Visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro
 - In particolare, visita gli elementi di un ARB in ordine
- Visita **pre-order** o **in profondità (depth-first)** – `void preOrder(Tree tree);`
 - Visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro
- Visita **post-order** – `void postOrder(Tree tree);`
 - Visita il sottoalbero sinistro, visita il sottoalbero destro, visita la radice



Visita in-order?

6, 13, 17, 27, 33, 42, 48

Visita pre-order?

27, 13, 6, 17, 42, 33, 48

Visita post-order?

6, 17, 13, 33, 48, 42, 27

Visita In-order

- In-order:
 - Visita: sottoalbero sx $\rightarrow$ radice $\rightarrow$ sottoalbero dx
- Caso base?
 - Albero vuoto
- Caso ricorsivo?
 - Visita degli sottoalberi
 - Prima e dopo della radice

```
void inOrder(Tree tree) {  
    if (tree == NULL) return;  
  
    inOrder(tree->left);           // 1°  
    /* Codice che visita la radice 2° */  
    inOrder(tree->right);          // 3°  
}
```

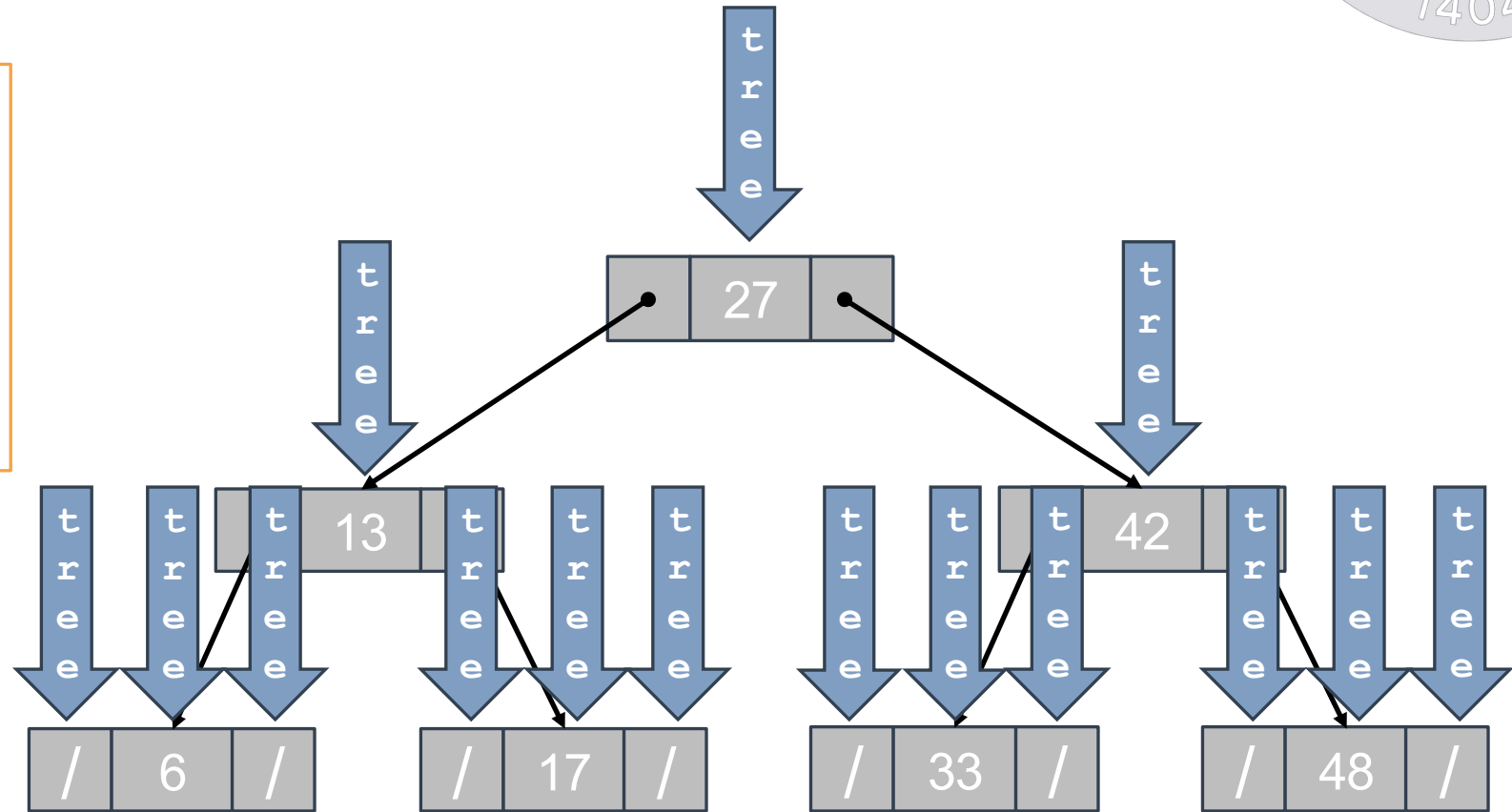


Visita In-order: Simulazione

```
void printInOrder(Tree tree) {  
    if (tree == NULL) return;  
  
    printInOrder(tree->left);  
    printf("%3d", tree->data);  
    printInOrder(tree->right);  
}
```

- Stampa:

6
13
17
27
33
42
48



Visita In-order: Esempi



- Esempi concreti di operazioni di visita in-order sono:
 - `void printInOrder(CharTree tree);`
 - Stampa gli elementi dell'albero in-order (vedi slide precedente)
 - `size_t size(Tree tree);`
 - Restituisce il numero di elementi dell'albero.
 - `size_t height(Tree tree);`
 - Restituisce l'altezza dell'albero.
 - `int sumIntTree(IntTree tree);`
 - Restituisce la somma degli elementi dell'albero.
 - `size_t count(Tree tree, Bool (*predicate)(T e));`
 - Restituisce il numero degli elementi dell'albero che soddisfano `predicate`
- Implementazioni lasciate come esercizio
- Dati n nodi e l'altezza dell'albero h , complessità computazionale?
 - Tempo:
 - $O(n)$
 - Spazio:
 - $O(h)$
 - Pessimo:
 - albero degenero $h = n-1$
 - Ottimo:
 - albero pseudo quasi completo $h = \log_2(n)$

Visita Pre-order e Post-order

- Pre-order (depth-first)
 - Visita: radice $\rightarrow$ sottoalbero sx $\rightarrow$ sottoalbero dx
 - Caso base? Ricorsivo?

```
void preOrder(Tree tree) {  
    if (tree == NULL) return;  
  
    /* Codice che visita la radice 1° */  
    preOrder(tree->left);           // 2°  
    preOrder(tree->right);          // 3°  
}
```

- Post-order
 - Visita: sottoalbero sx $\rightarrow$ sottoalbero dx $\rightarrow$ radice
 - Caso base? Ricorsivo?

```
void postOrder(Tree tree) {  
    if (tree == NULL) return;  
  
    postOrder(tree->left);           // 1°  
    postOrder(tree->right);          // 2°  
    /* Codice che visita la radice 3° */  
}
```

Visita Pre-order e Post-order: Esempi

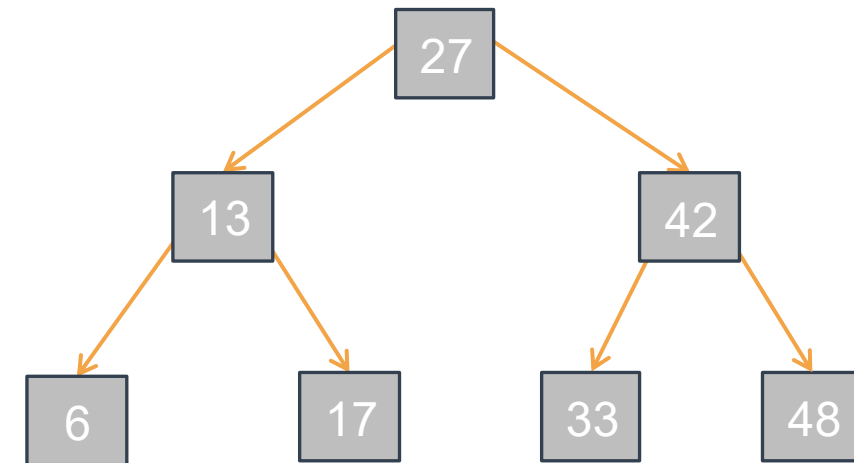
- Esempi concreti di operazioni di visita pre-order sono:
 - `void printPreOrder(CharTree tree);`
 - Stampa gli elementi dell'albero in pre-order (vedi slide precedente)
- Esempi concreti di operazioni di visita post-order sono:
 - `void printPostOrder(CharTree tree);`
 - Stampa gli elementi dell'albero in post-order (vedi slide precedente)
- E gli altri? `size()`, `height()`, `sumIntTree()`, `count()`?
 - Possono essere implementate usando sia in-order, pre-order, post-order
 - Complessità computazionale?
 - Rimane uguale: tempo $O(n)$, spazio $O(h)$
- Implementazioni lasciate come esercizio

Visita: Algoritmi Iterativi



Due algoritmi di visita iterativi che differiscono nell'ordine di visita radice e sottoalberi:

- Visita **in-ampiezza** (**breadth-first**) – `void breadthFirst(Tree tree);`
 - Visita l'albero per livelli iniziando dalla radice e i nodi di ciascun livello da sinistra a destra
 - Si appoggia ad una coda
- Visita **pre-order** o **in profondità** (**depth-first**) – `void depthFirst(Tree tree);`
 - Versione iterativa della visita per sottoalbero: radice poi sottoalbero sinistro e destro
 - Si appoggia ad una pila



Visita breadth-first?

27, 13, 42, 6, 17, 33, 48

Visita depth-first?

27, 13, 6, 17, 42, 33, 48

Visita Depth-first (Pre-order) Iterativa

- Depth-first: radice $\rightarrow$ sx $\rightarrow$ dx
 - Usiamo una pila s per appoggio: inseriamo nella pila i prossimi nodi da visitare
- Consideriamo (Invariante di Ciclo):
 - Se $!isEmpty(s)$, allora:
 1. I nodi di $tree$ già visitati sono quelli che precedono il nodo $top(s)$ nell'ordine pre-order e sono stati visitati nell'ordine pre-order;
 2. E s contiene, sotto a $top(s)$, in ordine decrescente di profondità, i figli destri degli antenati di $top(s)$ che non sono antenati di $top(s)$.
 - Se $isEmpty(s)$, allora:
 - Tutti i nodi di $tree$ sono stati visitati nell'ordine pre-order.

```
void depthFirst(Tree tree) {  
    if (tree == NULL) return;  
    VoidPtrStackADT s = mkVoidPtrStackADT();  
    push(s, tree);  
  
    while (!isEmpty(s)) {  
        Tree nodePtr = pop(s);  
        /* Codice che visita il nodo 1° */  
        if (nodePtr->right != NULL) {  
            push(s, nodePtr->right); // 2°  
        }  
        if (nodePtr->left != NULL) {  
            push(s, nodePtr->left); // 3°  
        }  
    }  
}
```

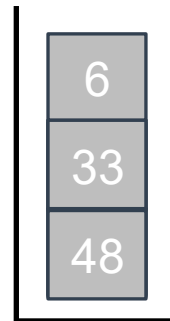
⚠ L'ordine è importante!

Visita Depth-first: Simulazione

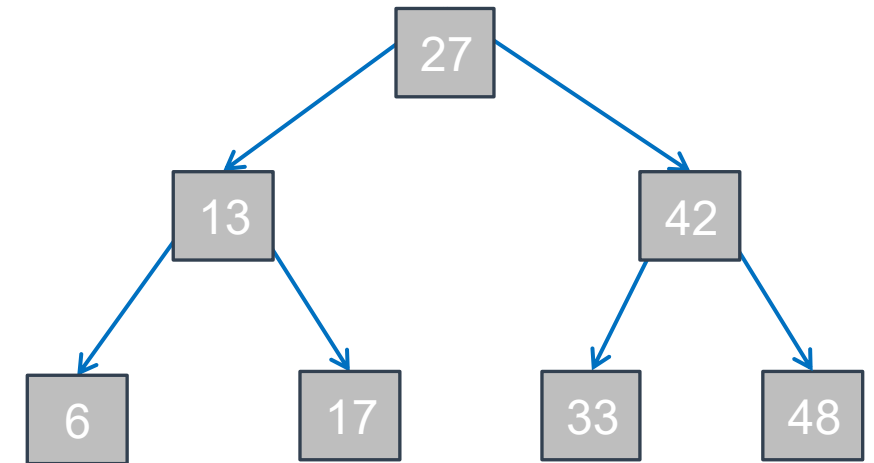
```
void depthFirst(Tree tree) {
    if (tree == NULL) return;
    VoidPtrStackADT s =
        mkVoidPtrStackADT();
    push(s, tree);

    while(!isEmpty(s)) {
        Tree nodePtr = pop(s);
        printf("%d", tree->data);

        if (nodePtr->right != NULL) {
            push(s, nodePtr->right);
        }
        if (nodePtr->left != NULL) {
            push(s, nodePtr->left);
        }
    }
}
```



- **Stampa:** 27 13 6 17 42 33 48



- **Complessità computazionale?**
 - Tempo:
 - $O(n)$
 - Spazio:
 - $O(h)$

Nota: identifico nodi tramite la radice

Visita Breadth-first (in Ampiezza) Iterativa



- Breadth-first: da sx a dx per livello (LSD)
 - Usiamo una coda (queue) per appoggio: inseriamo nella coda i prossimi nodi da visitare
- Consideriamo (Invariante di Ciclo):
 - Se !isEmpty(q), allora:
 1. I nodi di tree già visitati sono quelli che precedono il nodo front(q) nell'ordine LSD e sono stati visitati nell'ordine LSD
 2. $\text{livello}(\text{front}(q)) \leq \text{livello}(\text{rear}(q)) \leq \text{livello}(\text{front}(q)) + 1$
 - Se !isEmpty(q) e rear(q) != tree, allora:
 - q contiene nell'ordine LSD i nodi di tree che seguono nell'ordine LSD l'ultimo nodo visitato, fino al [nodo rear(q) che è il] figlio più a destra dell'ultimo dei nodi visitati che ha almeno un figlio.
 - Se isEmpty(q) allora:
 - Tutti i nodi di tree sono stati visitati nell'ordine LSD.

```
void depthFirst(Tree tree) {
    if (tree == NULL) return;
    VoidPtrQueueADT stack =
        mkVoidPtrQueueADT();
    enqueue(q, tree);

    while (!isEmpty(q)) {
        Tree nodePtr = dequeue(q);
        /* Codice che visita in nodo 1° */
        if (nodePtr->right != NULL) {
            enqueue(q, nodePtr->left); // 2°
        }
        if (nodePtr->left != NULL) {
            enqueue(q, nodePtr->right); // 3°
        }
    }
}
```

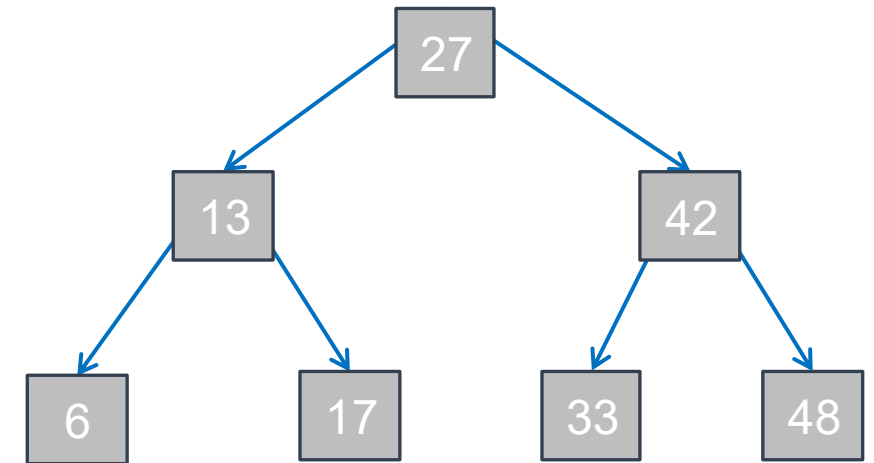
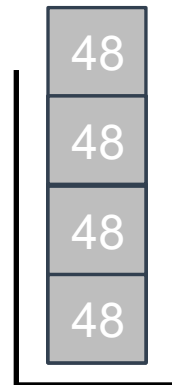
⚠ L'ordine è importante!

Visita Breadth-first: Simulazione

```
void breadthFirst(Tree tree) {
    if (tree == NULL) return;
    VoidPtrQueueADT q =
        mkVoidPtrQueueADT();
    enqueue(q, tree);

    while(!isEmpty(q)) {
        Tree nodePtr = dequeue(q);
        printf("%d", tree->data);

        if (nodePtr->right != NULL) {
            enqueue(q, nodePtr->left);
        }
        if (nodePtr->left != NULL) {
            enqueue(q, nodePtr->right);
        }
    }
}
```



- Stampa: 27 13 42 6 17 33 48

- Complessità computazionale?

- Tempo:

- $O(n)$

- Spazio:

- $O(n)$

Perchè?

Nota: identifico nodi tramite la radice

Visita Depth-first e Breadth-first: Esempi

- Esempi concreti di operazioni di visita depth-first sono:
 - `void printDepthFirst(CharTree tree);`
 - Stampa gli elementi dell'albero in pre-order (vedi slide precedente)
- Esempi concreti di operazioni di visita post-order sono:
 - `void printBreadthFirst(CharTree tree);`
 - Stampa gli elementi dell'albero in post-order (vedi slide precedente)
- E gli altri? `size()`, `height()`, `sumIntTree()`, `count()`?
 - Possono essere implementate usand sia depth-First che breadth-First
- Implementazioni lasciate come esercizio